

Assignment 2 Report

Name: K. Rishitha

Roll no: DA24B040

Q1: Single-stage vs. multi-stage Docker

Image	Dockerfile	Size
spam-api:naive	app/Dockerfile.naive (single stage)	1.47 GB
spam-api:v1	app/Dockerfile (multi-stage)	502 MB
Reduction		70.3%

Both images serve GET /healthz and POST /predict the same way. I tested this by curling both containers with the same spam/ham messages and got identical labels. That check is saved in evidence/q1_sizes.txt.

Why the multi-stage image is smaller. I used python:3.11-slim as the base for both Dockerfiles, so the size gap isn't coming from the base image at all—it's entirely from what each Dockerfile installs and leaves behind.

I confirmed this by shelling into both images and checking directly:

- **Compiler:** naive has gcc at /usr/bin/gcc; v1 has no compiler. I install build-essential/gcc/g++ in the builder stage in case scikit-learn/scipy need to compile anything, but the runtime requirements do not need a compiler.
- **Pandas:** naive has pandas fully installed. This is unnecessary in the final image because main.py only calls joblib.load() and predict(); pandas is used by train.py to read the CSV. The runtime requirements therefore do not install pandas.
- **Training files and cache:** naive still contains spam_dataset.csv, train.py, and a 92 MB pip cache. In v1, training happens in the builder stage and only the roughly 40 KB model.joblib crosses into the runtime stage.

So the naive build does everything in one place and keeps everything it touched, while the multi-stage build throws away the builder stage and only copies out the finished virtual environment plus the trained model.

Q2: Docker Compose + Redis cache

docker-compose.yml has two services: api, built from the multi-stage Dockerfile, and cache, using redis:7-alpine. Compose puts both on the same project network and gives each service name its own DNS entry, so main.py can use REDIS_HOST=cache without hardcoding an IP.

I checked this directly:

```
docker compose exec api python -c "import socket; print(socket.gethostbyname('cache'))"
```

which returned 172.19.0.2. docker compose exec cache redis-cli ping also returned PONG, confirming that both containers were reachable over the project network.

I also used depends_on with condition: service_healthy, so the API container waits for Redis to actually respond to a ping before starting. The startup logs showed:

```
[startup] cache enabled -> cache:6379 ttl=300s
[startup] model loaded from /app/model.joblib; version=1.0.0
```

For the caching logic, `main.py` hashes the incoming text and checks Redis first. On a miss it runs the model, stores the result with a 300-second TTL, and returns it. On a hit it skips the model and returns the cached label. The response body remains exactly `{"label": ...}`, while caching information is exposed through `X-Cache` and `X-Elapsed-Ms` headers.

A manual check showed:

- First request: `X-Cache: MISS, X-Elapsed-Ms: 17.553`
- Same request again: `X-Cache: HIT, X-Elapsed-Ms: 0.853`

Benchmark evidence. `scripts/bench_cache.py` sends 200 unique messages as misses and then replays the same 200 messages as hits.

Population	Mean	p50	p95
MISS (model runs)	1.789 ms	1.649 ms	2.409 ms
HIT (Redis only)	0.651 ms	0.538 ms	1.243 ms
Speedup	2.75×	3.07×	

All 200 labels matched exactly between the two runs. Redis statistics also backed this up: `{"cache":"enabled", "key": ...}`. The full benchmark output is in `evidence/q2_bench.txt`.

Compose vs. Kubernetes. Compose wires multiple containers together on one Docker host and gives them networking and startup dependencies. Kubernetes is designed for a cluster of nodes and provides scheduling, self-healing, rolling updates, and stable Services in front of pods. In short, Compose answers how containers work together on one machine, while Kubernetes manages workloads across a cluster.

Q3: Kubernetes Indexed Job, parallel shard validation

I used `parallelism = 4` and `completions = 8`, with CPU requests and limits both set to **500m**.

The validator is plain single-threaded Python using `csv.DictReader` and one regex check per row, so it cannot use more than one CPU. A 250-row shard finishes in well under a tenth of a second in the observed runs. I set requests equal to limits so each pod gets Guaranteed QoS.

The assignment assumes 2 nodes with 2 allocatable CPUs each. With 500m requested per pod, the theoretical cluster capacity is enough for more than four pods, but system components also consume resources. I therefore chose 4 pods as a conservative value: two pods per node, using 2000m total requested CPU, while leaving headroom for the Kubernetes system components. With 8 completions, this results in two waves of four.

I did not choose 8 because 8 times 500m would consume the entire nominal 4000m CPU capacity before accounting for system components. Some pods could remain pending, which would fail to demonstrate the chosen concurrency being reached.

The Job uses `restartPolicy: Never`. The manifest also passes `metadata.name` into `POD_NAME` and `spec.nodeName` into `NODE_NAME` through the Downward API. `JOB_COMPLETION_INDEX` identifies the shard automatically for an Indexed Job.

The concurrency evidence showed four pods running at the same time:

<code>shard-validation-0-bt4x8</code>	<code>1/1</code>	<code>Running</code>	<code>minikube-m02</code>
<code>shard-validation-1-bnfx4</code>	<code>1/1</code>	<code>Running</code>	<code>minikube-m02</code>
<code>shard-validation-2-gcw6m</code>	<code>1/1</code>	<code>Running</code>	<code>minikube</code>
<code>shard-validation-3-czqz6</code>	<code>1/1</code>	<code>Running</code>	<code>minikube</code>

The Job then completed all 8 completions in 56 seconds, consistent with two waves of four.

Why I used Kubernetes API logs instead of a shared volume. Minikube's default standard `StorageClass` is backed by `hostPath`, so a `PVC` maps to storage on a particular node. Since concurrent

Pods can land on both nodes, relying on a single-node volume would create a storage-placement problem and could force the workload onto one node. Using `kubectl logs` avoids that because logs are retrieved through the Kubernetes API from the kubelet hosting each pod.

The collected results were:

Shard	0	1	2	3	4	5	6	7	Total
Invalid rows	2	19	28	33	38	38	57	45	260
% of 250	0.8	7.6	11.2	13.2	15.2	15.2	22.8	18.0	13.0

These matched `GROUND_TRUTH.txt` exactly: 0/2, 1/19, 2/28, 3/33, 4/38, 5/38, 6/57, and 7/45.

If the cluster instead had 3 nodes with 2 CPUs each, I would raise parallelism to 8. With the same 500m request, the cluster would have enough capacity for all 8 shards to run together while still leaving system headroom. Since there are only 8 completions, increasing parallelism beyond 8 would not provide additional concurrency.

Q4: Deployment, self-healing and rolling updates

The Deployment runs **2 replicas** with requests of 250m CPU and 512Mi memory, limits of 500m CPU and 512Mi memory, and a readiness probe hitting `/healthz`. The rolling update strategy uses `maxUnavailable: 0` and `maxSurge: 1`. The Service is a NodePort selecting `app=spam-api`.

Self-healing. I deleted one running pod with `kubectl delete pod`. Before deletion, both pods were owned by the same ReplicaSet:

```
spam-api-865d7f44b5-wp8wh <- ReplicaSet/spam-api-865d7f44b5
spam-api-865d7f44b5-wrfsn <- ReplicaSet/spam-api-865d7f44b5
```

After deleting the first pod, Kubernetes created a replacement:

```
4s    Normal    Killing    pod/spam-api-865d7f44b5-wp8wh
4s    Normal    Scheduled  pod/spam-api-865d7f44b5-jpbmd
2s    Normal    Started    pod/spam-api-865d7f44b5-jpbmd
```

The ReplicaSet controller is responsible for maintaining the desired number of replicas. The Deployment manages the ReplicaSet, while the ReplicaSet continually compares the current number of matching pods with the desired count and creates a replacement when there is a shortfall.

Rolling update. I built `spam-api:v2` from the same Dockerfile with `APP_VERSION=2.0.0`, so `/healthz` reports version 2.0.0. I then updated the Deployment image and followed the rollout using `kubectl rollout status` and `kubectl rollout history`.

The new ReplicaSet ended at 2 ready replicas while the older ReplicaSets were scaled to zero:

NAME	DESIRED	CURRENT	READY
spam-api-6ddfb6b7fb	0	0	0
spam-api-749cf67dd7	2	2	2
spam-api-865d7f44b5	0	0	0

With `maxUnavailable: 0`, Kubernetes brings a new pod up and waits for its readiness probe before removing an old pod. This keeps the desired number of ready replicas available during the rollout.

If this had been a Job instead of a Deployment, there would be no ReplicaSet-based rolling update, rollout history, or rollback mechanism. A new version would require creating a new Job, because Jobs are designed for run-to-completion work rather than continuously serving requests.

More generally, a Deployment fits a workload that should stay running and answer requests, so it maintains a desired number of replicas and supports rolling updates. A Job fits work that should

finish, so successful pod completion is the desired end state. This is why the spam API belongs in a Deployment while the shard validator in Q3 belongs in a Job.